

A Configurable Test Case Prioritization Technique for Early Fault Detection and Low Test Case Spreading

Wesley N. M. Torres
Federal University of Campina
Grande
Brazil
wesley.torres@splab.ufcg.edu.br

Everton L. G. Alves
Federal University of Campina
Grande
Brazil
everton@computacao.ufcg.edu.br

Patrícia D. L. Machado
Federal University of Campina
Grande
Brazil
patricia@computacao.ufcg.edu.br

ABSTRACT

Developers often use test suites as safety nets to avoid functionality regression. However, regression testing can be costly and time-consuming. Test case prioritization (TCP) techniques try to reduce this burden by reordering the tests of a given suite to achieve a certain testing goal. The literature presents a great number of TCP techniques. Most works evaluate the performance of TCP techniques by using the rate of test cases that fail per fault (APFD). However, other aspects can be considered when evaluating prioritization results. For instance, the ability to reduce the spreading of failing test cases (M-Spreading), since a better grouping often provides more information regarding faults. This paper proposes *Additional-Spreading*, a configurable test case prioritization strategy that allows testers to favor either APFD or M-Spreading. In a study with open-source projects we evidence the configuration power of *Additional-Spreading* and that it can provide results similar to two traditional techniques (*Total* and *Additional*) when properly configured.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**.

KEYWORDS

prioritization, test case, metric, APFD, M-Spreading

ACM Reference Format:

Wesley N. M. Torres, Everton L. G. Alves, and Patrícia D. L. Machado. 2022. A Configurable Test Case Prioritization Technique for Early Fault Detection and Low Test Case Spreading. In *36th Brazilian Symposium on Software Engineering (SBES'22)*, October 03–07, 2022, Uberlândia, Brazil. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3555228.3555231>

1 INTRODUCTION

Regression test suites address functionality regression [17]. Developers often apply them to make sure the behavior of previous software versions remains valid after code edits. For instance, Fowler mechanics for refactorings combine small edits with the execution

of regression suites [6]. However, regression testing is known as a costly and time-consuming activity. Especially in agile projects, it might be impractical to rerun the whole test suite after each code change [17].

Test Case Prioritization (TCP) techniques propose a new test execution order to achieve some testing goal. By using a prioritized test suite, developers may run the top test cases according to the available resources. Test case prioritization is known to provide productivity gains and avoid bugs [29].

The prioritization problem was first defined by Rothermel et al. [23] as follows:

Given: T , a test suite; PT , the set of permutations of T , and f , a function from PT to real numbers.

Problem: Find $T' \in PT$ such as $\forall T'' \in PT \bullet T'' \neq T' \rightarrow (f(T') \geq f(T''))$, where PT represents the set of possible orderings of T , and f is a function that calculates the best results when applied to any ordering.

There is a great number of TCP techniques [7, 16, 17, 25]. They vary by the prioritization method, heuristics, context, and/or testing goal. For instance, the *Total* strategies [25] use the heuristics that test cases with most coverage (e.g., statements) are more likely of detecting faults and should be run first. On the other hand, the *Additional* strategies [25] reorder the tests based on code elements not yet covered by other tests.

Often, TCP techniques intend to fasten fault detection. The most common metric to evaluate the quality of a TCP is the APFD (Average Percentage Faults Detected) [23]. It calculates the weighted average of the percentage of faults detected for a given test suite order. APFD values may guide a tester to choose a TCP technique. However, by considering only the APFD, other important evaluation aspects might be neglected.

A tester often runs only the top test cases of a prioritized suite. The number of tests to run depends on practical constraints (e.g., shortage of time). However, if the top test cases gather only part of the failing ones, this tester might have a hard time understanding and/or locating the faults. Yoo et al. [30] found that a tester often needs a set of failing test cases for the same type of fault since it helps fault localization. Moreover, after a bug fix, a developer might want to run all failing tests to see if the fix generalizes. Thus, besides early fault detection, we believe that a good TCP technique should also reduce related failing test cases' spreading.

Alves et al. [1] propose *F-Spreading*, a metric for evaluating TCP based on the spreading of test cases that reveal a fault. Later, this metric evolved to *M-Spreading* [28], which captures the spreading of test cases that reveal different faults. Moreover, they ran an empirical study that evaluated how twelve of the most popular TCP

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBES'22, October 03–07, 2022, Uberlândia, Brazil

© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9735-3/22/10...\$15.00
<https://doi.org/10.1145/3555228.3555231>

```

1 public static String genHash(String value, String algorithm){
2     Number key = NumberUtils.createNumber(value);
3     try{
4         MessageDigest md = MessageDigest.getInstance(algorithm);
5         md.update(key.toString().getBytes());
6         return new BigInteger(1, md.digest()).toString(16);
7     }catch (NoSuchAlgorithmException e){ return null; } }

```

```

1 public static Number createNumber(final String str) throws
    NumberFormatException{
2     if(str == null) return null;
3     if(str.length() == 0) throw new NumberFormatException("A blank
    string is not a valid number");
4     final String[] hex_prefixes = {"0x", "0X", "-0x", "-0X", "#", "-#"};
5     int pfxLen = 0;
6     ...
7     if (pfxLen > 0){
8         final int hexDigits = str.length() - pfxLen;
9         if(hexDigits > 16){ return createBigInteger(str); }
10        if(hexDigits > 8){ return createLong(str); }
11        return createInteger(str);
12    }
13    return createBigDecimal(str); }

```

```

1 public static Integer createInteger(final String str){
2     if(str == null){ return null; }
3     return Integer.decode(str); }

```

(c) Method that decodes an integer value from a string. It contains a fault since it does not check whether the given string represents a valid integer.

Figure 1: Code snippets from the Lang project that contains a fault.

techniques behave when observing both the rate of fault detection (APFD) and the spreading of failing test cases (M-Spreading) for one or more faults [28]. They found several cases where a prioritization generated good APFD results but with very bad spreading (M-Spreading).

In this work, we present *Additional-Spreading*, a novel TCP technique that allows a tester to customize the prioritization and guide it to favor different testing goals: fastening fault detection (APFD); better test case grouping (M-Spreading); or a balancing scenario. Then, we run a series of empirical studies to show the configuration power of *Additional-Spreading*. We compare it to a group of well-known TCP techniques, and we provide guidelines on how developers should use it.

2 MOTIVATIONAL EXAMPLE

Consider the following fault from Defect4j [13] that is related to the Lang project (Figure 1): methods `genHash` and `createNumber` generate hash codes from keys that can be represented in several numerical bases. `createNumber` calls `createInteger` (line 11), however, this method does not check whether the input value is within the range of valid numbers for a specific base.

In a suite with 300 test cases, this is a subtle fault revealed by two tests (Figure 2). Though both detect the fault, together they provide better information for understanding/fixing the fault. Both line 4 (Figure 2(a)) and line 8 (Figure 2(b)) provide inputs that overflow the integer limits. Depending on the applied prioritization, those tests might not be executed. For instance, a given TCP strategy may place them in positions 3 and 200, respectively. This configuration would provide an excellent APFD result (0.99). However, if a tester decides to run the top 100 test cases, the fault would be detected, but a single test case would fail (`testGenHash`).

```

1 public void testGenHash(){
2     assertEquals("bcabf6a179d32acfd669938266c6b92", Auth.genHash("0x8000", "
    MD5"));
3     assertEquals("9ab98b1f1e647c65ebdf670363bc813", Auth.genHash("0x80000000
    ", "MD5"));
4     assertEquals("d2f7d27c4d72f6d5b74d14b859e3c77d", Auth.genHash("0xFFFFFFFF
    ", "MD5"));
5     assertEquals("c4ca4238a0b923828dc509a6f75849b", Auth.genHash("0x000001",
    "MD5")); }

```

```

1 public void testCreateInteger(){
2     assertEquals(Integer.valueOf(0x8000), NumberUtils.createInteger("0x8000"));
3     assertEquals(Integer.valueOf(0x80000), NumberUtils.createInteger("0x80000
    "));
4     assertEquals(Integer.valueOf(0x800000), NumberUtils.createInteger("0
    x800000"));
5     assertEquals(Integer.valueOf(0x8000000), NumberUtils.createInteger("0
    x8000000"));
6     assertEquals(Integer.valueOf(0x7FFFFFFF), NumberUtils.createInteger("0
    x7FFFFFFF"));
7     assertEquals(Integer.valueOf(0x80000000L), NumberUtils.createInteger("0
    x80000000"));
8     assertEquals(Integer.valueOf(0xFFFFFFFFL), NumberUtils.createInteger("0
    xFFFFFFFF")); }

```

(b) Generation hash code to user

Figure 2: Tests from the Lang suite.

Fastening fault detection is important (high APFD). However, a single failing test case might not be enough to help understand/locate the fault. In our example, `testGenHash` does not directly cover the faulty method (`createInteger`), i.e., fault detection is due to an indirect sequence of actions. This scenario is not easily noticeable by a tester [2]. `testCreateInteger`, on the other hand, tests a series of acceptable values for `createInteger`, which may be more informative about the fault.

When we consider a spreading measure such as *M-spreading*, we can see that the applied prioritization was not very effective (*M-spreading* = 0.66). Therefore, we can say that a good prioritization should propose an execution order that combines fast detection (high APFD) with a low spreading of failing test cases per fault (*M-spreading*).

3 BACKGROUND

Prioritized suites can be evaluated using different metrics [22]. Here, we focus on two of them: APFD and M-Spreading.

3.1 The Average Percentage of Faults Detected (APFD)

This metric was proposed by Rothermel et al. [23] and captures how fast a prioritized suite can detect a series of faults. The earlier all faults are detected, the higher the APFD. Equation 1 presents how APFD is calculated. Let T be a test suite containing n test cases, F be a set of m faults revealed by T , n the number of test cases, and TF_i be the index of the first test case in ordering T of T that reveals fault i .

$$APFD = 1 - \frac{(\sum_{i=1}^m TF_i)}{nm} + \frac{1}{2n} \quad (1)$$

Suppose two faults (f_1 and f_2) are detected by a suite with 10 test cases $\{TC_1, TC_2, TC_3, TC_4, TC_5, TC_6, TC_7, TC_8, TC_9, TC_{10}\}$. f_1 is detected by TC_2 and TC_8 , and f_2 is detected by TC_9 . Now, suppose that a tester applied two TCP techniques (T_1 e T_2) that generate

the following prioritized suites $S1 = [TC_7, TC_1, TC_6, TC_2, TC_3, TC_4, TC_5, TC_8, TC_9, TC_{10}]$ and $S2 = [TC_1, TC_2, TC_5, TC_4, TC_6, TC_7, TC_8, TC_3, TC_9, TC_{10}]$, respectively. By applying Equation 1 to $S1$ and $S2$, we have the following APFD values: 0.45 (45%) and 0.6 (60%), respectively. Thus, considering APFD values, the tester would select $T2$ as the best TCP technique.

3.2 The Spreading of Failing Test Cases (M-Spreading)

The *M-Spreading* metric, proposed by Torres et al. [28], extends *F-Spreading* [1]. It measures the spreading of all related fault-revealing test cases in a prioritized suite, based on the assumption that test cases that fail due to a giving fault often provide good information for helping fault understanding and localization [30]. Therefore, as more clustered related faults are in a prioritized suite, lower it is its *M-Spreading*.

M-Spreading is calculated by the average *F-Spreading* for each individual fault (Equation 2), where N is the number of test cases; f is the number of faults; $F\text{-spreading}_i$ is the *F-Spreading* for i^o fault. *F-Spreading* is calculated according to Equation 3, where N is the number of test cases in a prioritized suite; m is the number of tests that failed; TF is the sequence of the positions of the failing test cases; and TF_i is the position of the test case i in the prioritized set.

$$M\text{-Spreading} = \frac{\sum_{i=1}^f F\text{-Spreading}_i}{f} \quad (2)$$

$$F\text{-Spreading} = \begin{cases} 0, & \text{if } m < 2 \\ (\sum_{i=2}^m TF_i - TF_{i-1}) * \frac{1}{N}, & \text{if } m \geq 2 \end{cases} \quad (3)$$

For the example in Section 3.1, the *M-Spreading* values for $S1$ and $S2$ are 0.25 and 0.35, respectively. Thus, $T1$ has the lowest spreading.

4 THE ADDITIONAL-SPREADING TECHNIQUE

Torres et al. [28] found that TCP techniques may present diverging results depending on the evaluation metric used (APFD or *M-Spreading*). For instance, the *Total* strategy performed better for *M-Spreading*, but worse for APFD. Therefore, we can say there is no final technique that covers both fastening fault detection (high APFD) and low spreading of failing test cases (low *M-Spreading*). Moreover, a tester's goal for prioritization may vary throughout time. In a moment, due to time constraints, a tester might want to favor fastening fault detection (APFD). At another time, she might wish to know more about a fault during a bug fixing activity (better *M-Spreading*). Finally, a tester might want a prioritization that balances both goals (APFD and spreading). In this sense, we propose *Additional-Spreading* a configurable TCP technique. Its main goal is to provide a prioritization that can be adapted according to the tester's need.

4.1 The Prioritization strategy

Additional-Spreading aims to enable the bridging of two testing goals: (i) fastening the detection of faults and (ii) reducing the spreading of related test cases. Therefore it is defined in terms

of the two prioritization strategies that better performed in [28]: *Additional* (i), and *Total* (ii).

With the *Additional* strategy Torres et al. found the best APFD results. Its effectiveness was also highlighted by other studies [31]. Therefore, the *Additional-Spreading* prioritization algorithm reuses the main ideas of the *Additional* strategy: starts with the test case with most coverage; and each round, it selects the test that covers most code elements that were not yet exercised.

Although often providing an effective APFD, the *Additional* strategy was greatly outperformed by the *Total* regarding the spreading of related test cases (*M-Spreading*). *Total* orders the test cases according to their coverage values. Therefore, with *Additional-Spreading*, we enable in the same prioritization a way to mix the *Additional* and *Total* strategies.

Additional-Spreading is represented by Algorithm 1 and its auxiliary functions (Algorithms 2, 3, and 4). It receives as input a test suite, its coverage matrix, and two configurable thresholds (parameters): similarity (α), and cutting (β). Those parameters were introduced to enable the configurable aspect. In summary, at each prioritization round, the strategy selects the test cases that cover most not yet exercised code parts (similar to *Additional*). Then, it groups test cases that are similar to the recently selected. This is done by using a distance function and the similarity threshold (α). The selected group is then ordered based on the number of covered code elements (similar to *Total*). After ordering, we cut the group of similar tests by using the cutting threshold (β) to avoid adding a great number of tests in a single round. The tests that remain after cutting are added to the prioritized suite (ordered). This process is repeated until all tests are prioritized.

Algorithm 1 Additional-Spreading(testSuite, coverageMatrix, α , β)

```

1: prioritizedSuite ← {};
2: testSimMatrix ← calcSimMatrix (testSuite, similarityFunction);
3: unitsNotCoveredBySuite ← {}
4: while prioritizedSuite.size() < testSuite.size() do
5:   maxTestCovering ← selectMaxNonCoveredYet(testSuite, coverageMatrix, prioritizedSuite);
6:   prioritizedSuite ← prioritizedSuite ∪ maxTestCovering;
7:   currentSimilarTests ← testsSimilarOf(maxTestCovering, testSimMatrix,  $\alpha$ );
8:   orderTestsCoverage(currentSimilarTests);
9:   cuttedTests ← getFirstNTests(currentSimilarTests,  $\beta$ );
10:  prioritizedSuite ← prioritizedSuite ∪ cuttedTests;
11: end while
12: return prioritizedSuite;
```

Algorithm 2 calcSimMatrix (testSuite, simFunction)

```

1: lenMatrix ← #testSuite;
2: simMatrix ← [lenMatrix][];
3: for  $i \leftarrow 0$  to lenMatrix do
4:   for  $j \leftarrow i$  to lenMatrix do
5:     simMatrix[i][j] ← simFunction(testSuite[i], testSuite[j])
6:   end for
7: end for
```

4.2 The Thresholds

For discussing the role of the similarity (α) and cutting (β) thresholds in *Additional-Spreading*, consider a the following test suite:

Algorithm 3 testsSimilarOf(maxTestCover, testSimMatrix, α)

```

1: lenMatrixRows  $\leftarrow$  #testSimMatrix.rows
2: testsSimilarOf  $\leftarrow$  {}
3: for  $i \leftarrow 0$  to lenMatrixRows do
4:   if testSimMatrix[maxTestCover][ $i$ ]  $\geq \alpha$  then
5:     testsSimilarOf  $\leftarrow$  testsSimilarOf  $\cup i$ 
6:   end if
7: end for

```

Algorithm 4 getFirstNTests(currentSimilarTests, β)

```

1: maxTests  $\leftarrow$  #currentSimilarTests *  $\beta$ 
2: return currentSimilarTests.firstN(maxTests)

```

$\{t_1, t_2, t_3, t_4, t_5\}$. Table 1 presents the coverage matrix for this suite. This matrix relates test cases to the code they cover (e.g., statements). An $\checkmark$ indicates that test t_i covers code unit u_j .

Table 1: Coverage matrix for the test cases of our running example.

Tests/Code Units	u_1	u_2	u_3	u_4	u_5	u_6	u_7	u_8
t_1	$\checkmark$				$\checkmark$		$\checkmark$	
t_2						$\checkmark$		$\checkmark$
t_3	$\checkmark$		$\checkmark$	$\checkmark$	$\checkmark$			
t_4	$\checkmark$	$\checkmark$		$\checkmark$			$\checkmark$	$\checkmark$
t_5	$\checkmark$		$\checkmark$			$\checkmark$		

4.2.1 The Similarity Threshold (α). Since *Additional-Spreading* combines the *Additional* and *Total* strategies, it reuses part of the *Additional* algorithm and includes elements to enable a better test case grouping. Oliveira Neto et al. [21] state that tests that exercise similar code parts tend to reveal the same faults. Therefore, *Additional-Spreading* applies similarity to select tests that resemble the one selected at the beginning of the prioritization round (Algorithm 1 - line 2).

To work with similarity, we adapt the use of distance functions. A distance function measures how similar, or different, two strings, or sets, are [3]. Distance functions have been used in different contexts (e.g., [4, 18, 26]). There are several distance functions (e.g., [9], [15]). For instance, the Jaccard index [20] measures similarity between finite sample sets. While our prioritization strategy is not tied to a specific distance function, for the sake of simplicity and evaluation, we use Jaccard for test case similarity: $Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}$.

We adapted the use of distance functions for test case similarity as in [3]. Each test case is represented by a string of '0's and '1's, where a '1' represents a code element covered by this test, '0' otherwise. Thus, a distance function receives two strings referring to test case coverage and informs how similar the strings (the tests) are. Back to our running example, the correspondent strings for test cases t_4 and t_5 are "11010011" and "10100100", respectively. Therefore, their Jaccard index is $\frac{1}{7} = 0.14$. Table 2 presents the similarity matrix for all tests.

The similarity threshold (α) is a value between 0 and 1 and is provided by the tester as an input factor. It is used to decide when two test cases are considered similar during *Additional-Spreading* prioritization. For instance, being $\alpha = 0.5$, and t_4 the test case selected in line 5, t_5 would not be included in the similar set (line 7).

Table 2: Similarity matrix for the test cases of our running example.

Tests/Tests	t_1	t_2	t_3	t_4	t_5
t_1	1	0	0.4	0.33	0.2
t_2	-	1	0	0.33	0.25
t_3	-	-	1	0.29	0.5
t_4	-	-	-	1	0.14
t_5	-	-	-	-	1

4.2.2 The Cutting Threshold (β). Although tests that cover similar code elements tend to reveal the same faults, which may favor M-Spreading, a prioritized suite should not neglect the goal of fastening fault detection (APFD). To work as a balancing factor, we cut the similarity set using a giving threshold (β). This value works to avoid scenarios where a great number of test cases are prioritized in a single round due to high test case similarity, which goes against the traditional prioritization strategy of the *Additional* techniques.

A Cutting Threshold is also a value between 0 and 1 and represents how much of the similar set is added to the prioritized suite at once. Our prioritization can be adapted according to the tester's goals. Back to our running example, suppose a tester chooses the following thresholds $\alpha = 0.3$ and $\beta = 0.5$. Considering t_4 , our prioritization would select as similar tests t_1 and t_2 ($similarity > 0.3$). However, as $\beta = 0.5$, only 50% of the similar tests are in fact added to the prioritized suite in the current round. Therefore, only t_1 will be added, since it covers more code elements than t_2 .

Our prioritization strategy was designed to be adaptive, therefore, we do not set any specific values for neither α nor β . However, we run an empirical investigation to understand the strategy sensibility to those factors (Section 5) and we provide guidelines on how to use them (Section 7).

4.3 Additional-Spreading Example

To better understand our prioritization strategy, let's run it step-by-step using our original example. Suppose the original suite ($\{t_1, t_2, t_3, t_4, t_5\}$) needs prioritization. A tester decides to run *Additional-Spreading* with $\alpha = 0.3$ $\beta = 0.5$. The first test case to be selected is the one that covers most code parts t_4 (Table 1). Then, it selects the tests similar to t_4 (Table 2) according to the similarity threshold (> 0.3), which gives us the similar set $\{t_1, t_2\}$. From this set, we select only half to be prioritized in this round ($\beta = 0.5$). In this case, t_1 would be selected, since it covers more code elements. Till now, the prioritized set is $[t_4, t_1]$. In the second round, we select the test case that covers more untested code units. Since the only two uncovered code units are u_3 and u_6 , the test selected is t_5 . Then, the similar set for t_5 is $\{t_3\}$, which will be added to the suite: $[t_4, t_1, t_5, t_3]$. Finally, t_2 is added resulting in $[t_4, t_1, t_5, t_3, t_2]$.

5 EXPLORING ADDITIONAL-SPREADING AND ITS PARAMETERS

Additional-Spreading applies a strategy that merges concepts of two well-known TCP techniques, *Additional* and *Total*. It is greatly influenced by its input parameters: similarity (α) and cutting thresholds (β). By changing their values, a tester may guide prioritization

to favor one of two goals: fastening fault detection (APFD) or reducing the spreading of fault revealing test cases (M-Spreading). To investigate the sensibility of *Additional-Spreading*, we run an empirical study.

5.1 Study Goal, Objects, Faults

The goal of this study is to understand the impact of different values of α and β on *Additional-Spreading* prioritization results considering both APFD and M-Spreading. The following research questions guide our investigation:

- **RQ1:** How does *Additional-Spreading* behave considering different combinations of α and β ?
- **RQ2:** What is the ideal combination of α and β for a balanced prioritization?

This study design is inspired by a related work [17] and reuses part of its implementations. We use eight Java projects from Github as objects. These projects have also been used in other empirical studies [28, 32]. From the project's repositories, we collect a set of stable versions for each project. Each version includes a JUnit test suite. Each pair of versions comprises a distance of at least 30 commits. For each version, we create different fault groups. Each fault group comprises five randomly injected mutant faults and each group can be detected by at least one test. All mutants are created by using the PIT tool¹. Mutant faults have been widely used and are found to be representative of real faults [5, 11].

Table 3 summarizes the artifacts used, where *Ver* is the number of versions, *MinS* and *MaxS* are the minimum and maximum number of lines of code (KLOC), *MinT* and *MaxT* are the minimum and maximum number of tests, and *Ft* the maximum number of faults present in all versions.

Table 3: Summary of the artifacts used in our study.

Proj	Ver	MinS	MaxS	minT	MatT	Ft
jasmine-maven	5	1.640	4.348	7	118	136
java-apns	8	1.362	3.839	15	87	82
jopt-simple	4	6.636	8.569	394	657	335
la4j	9	8.064	12.555	172	625	1013
scribe-java	8	2.497	5.957	38	99	112
vraptor-core	5	31.176	32.997	985	1.124	1463
assertj-core	8	55.443	67.282	4.055	5.269	1711
metrics-core	6	11.447	12.546	270	318	721

5.2 Procedure

Since similarity and cutting thresholds are inputs to *Additional-Spreading* and we want to investigate their impact on the prioritization, we vary both parameters within their limits ([0..1]) considering steps of 0.05 ($\{0, 0.05, \dots, 0.95, 1.0\}$). For similarity, we use the Jaccard function. Then, for each test suite, we run the prioritization algorithms and collect the correspondent APFD and M-Spreading results. *Additional-Spreading* is not restricted to a single coverage data type. Therefore, we consider three variations of *Additional-Spreading*, based on different coverage level data: *Additional-Spreading-statement*, *As Additional-Spreading-branch*,

and *Additional-Spreading-method*. To assure statistical validity, and minimize the impact of random aspects of the TCP techniques, we run each technique 1000 times. The results discussed in the following sections consider the average of those executions. Our implementations are available in our website².

5.3 Results and Analysis

5.3.1 RQ1: How does *Additional-Spreading* behave considering different combinations of α and β ? For *Additional-Spreading-statement*, Figures 3 and 4 present the distributions for the APFD and M-Spreading, respectively. A full version of the collected data can be found on our website³. The x-axis refers to the cutting values (β), the label of the box to similarity values (α), and the y-axis to the metric results (APFD or M-Spreading). For instance, the box in the top-left corner of Figure 3 shows the distribution of the APFD values for each cutting value (x-axis) when a similarity of 0% is considered for (α). For better comparison, in this analysis, we inverted the M-Spreading results ($1 - \text{M-Spreading}$). Therefore, the closer to one an M-Spreading value is, the better.

When we observe the M-Spreading distributions, we can find an ascending behavior as the cutting and similarity values grow. However, this tendency stops, when similarity values above 75% are used. For the APFD distributions, we find an opposite behavior, as the cutting values grow, there is a tendency for lower APFD results. Again, for similarity values above 75%, we cannot see considerable variations. This behavior is expected since high similarity values tend to make our strategy too restrictive when selecting candidates to prioritize. Moreover, goes according to Jiang et al. findings regarding defect patterns [11].

One may conclude that the similarity threshold does not have much impact on the M-Spreading results, since the median values do not show much improvement. However, it is important to highlight that the cutting value is applied only to the set of similar test cases. Therefore, we believe the similarity is an indirect factor to the collected data.

Thus, we can answer RQ1 by stating that *Additional-Spreading* provides a trend when different values are applied for α and β . We find worse APFD values as β goes higher, while M-Spreading values improve. Moreover, results seem to stop improving when α gets higher than 0.75 (75%).

We plotted the metrics curves based on the medians (Figure 5). As we can see, in several cases *Additional-Spreading* is able to greatly improve the M-Spreading results without much depreciation the APFD (e.g., Figure 5 for similarity below 75%). Moreover, similarity values above 75% often lead to extreme behavior with near-constant lines that never touch. In those cases, if a balance between APFD and M-Spreading is desired, it will never be achieved. Finally, we could also observe little loss for APFD, but improvements for M-Spreading, as the cutting threshold grows.

A similar trend was found when we observe the projects individually, except for **la4j**. For this project, *Additional-Spreading* performs quite poorly regarding APFD, differing from the others. As the cutting values grow, the APFD results are greatly affected.

¹<http://pitest.org/>

²<https://github.com/WesleyMarques/tcp-experiment/blob/master/experimento-techniques/src/main/GreedyAdditionalSimilarity.java>

³<https://sites.google.com/view/evaluating-tcp-spreading>

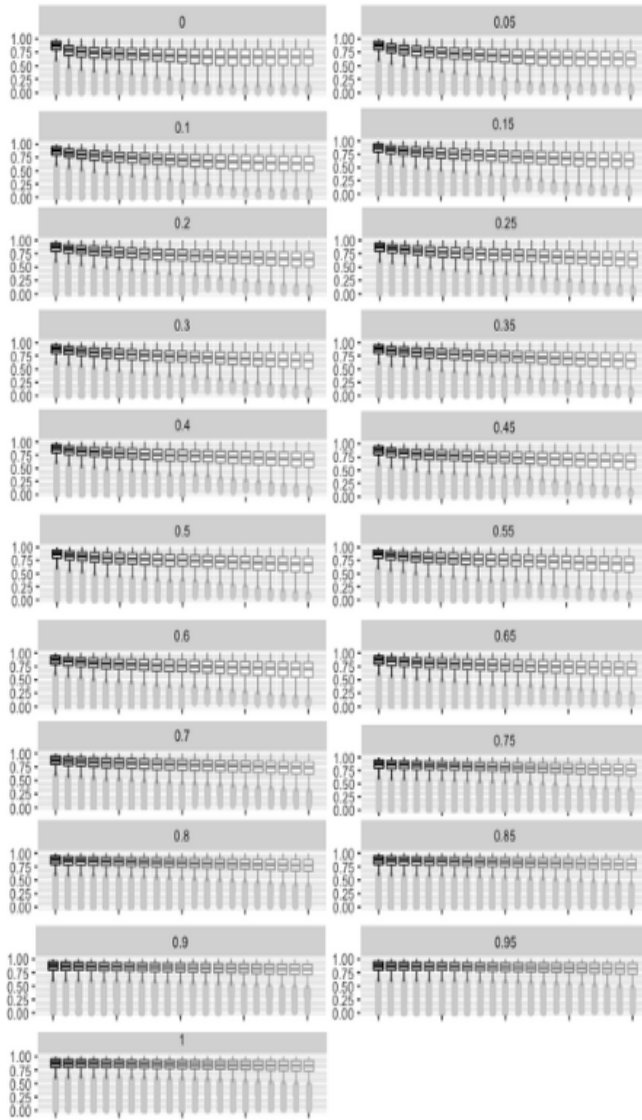


Figure 3: APFD for all values of α and β for Additional-Spreading-statement.

We perform a side investigation to reason about this result, and we observe that this project's test suite brings an interesting scenario: low coverage test cases detect a great number of faults, while high coverage test cases detect few faults. For instance, the test case `org.la4j.decomposition.EigenDecompositorTest.testDecompose` covers only 210 code units and detects 647 faults. On the other hand, test `org.la4j.vector.dense.BasicVectorTest.testAccess_4` covers 1509 code units, but detects only six faults. This unbalanced scenario was not found in other projects. Since *Additional-Spreading* is a coverage-based TCP, it relies on the assumption that test coverage often reflects fault detection. Moreover, as we reuse the projects and infrastructure from [17], we do not control the applied fault inject process, which may have led to this unbalanced scenario due to

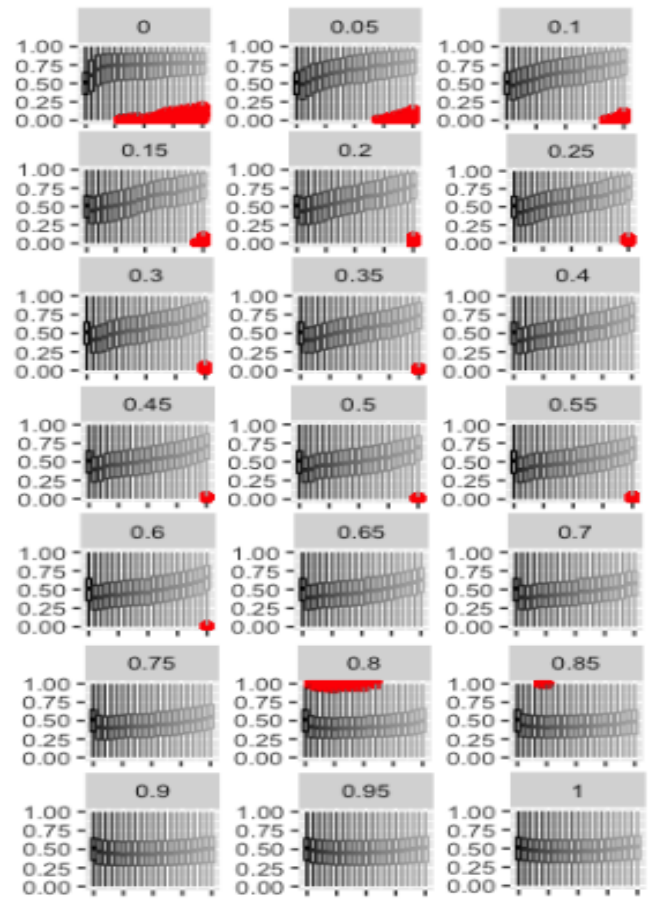


Figure 4: M-Spreading for all values of α and β for Additional-Spreading-statement.

specific characteristics of the project and/or test suite. Therefore, we consider the **la4j** an outlier for our investigation, and its results should be analyzed separately.

5.3.2 RQ2: What is the ideal combination of α and β for a balanced prioritization? We believe that a good prioritization balances fast detection (APFD) with low test case spreading (M-Spreading). Therefore, for each system used in our study, we plot the metrics curves for a series of different values of α and β and we select ones where the metric curves are the closest. It is important to highlight that by ideal here we mean the configuration where APFD and M-Spreading find a balance. Moreover, we perform this analysis considering each project individually since α and β can be influenced by specific aspects such as a project' and its suite' characteristics.

Table 4 presents the found ideal values for a balanced prioritization. For instance, considering the *Vraptor* project, its ideal configuration for a balanced prioritization using *Additional-Spreading* would be $\alpha = 0.15$ and $\beta = 0.65$.

Note that those configuration values are specific to the projects. In practice, if a balancing scenario is desirable, before using *Additional-Spreading*, we recommend a tester to run *Additional-Spreading* using a few different values for α and β to have a sense on the

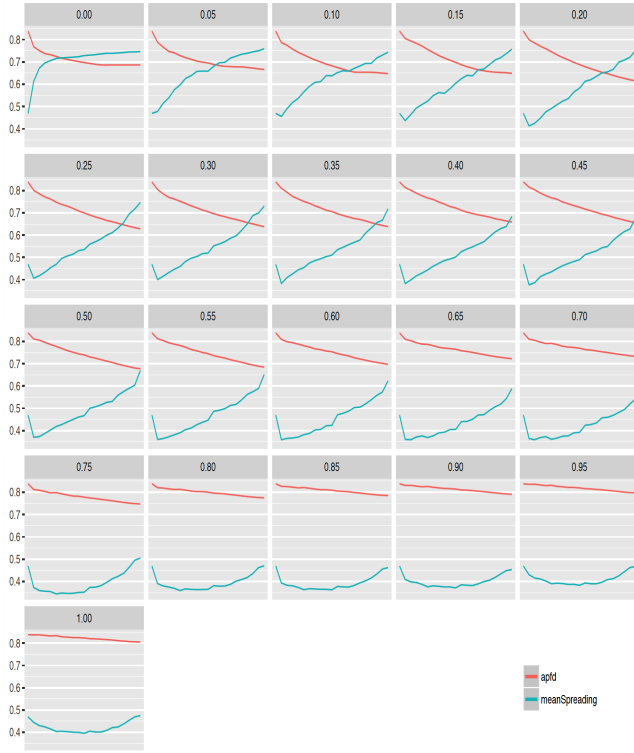


Figure 5: APFD and M-Spreading curves (considering median values) for Additional-Spreading-statement.

balancing configuration. these values can be later tuned. However, since our study works with a range of projects with different sizes and characteristics, a tester might be inspired by the numbers from Table 4. Finally, the next sections provide insights on how to first set α and β .

Table 4: Ideal configuration for α and β when aiming to use Additional-Spreading for a balanced prioritization. These results consider the three variations of Additional-Spreading (statement, branch, and method).

Projects	statement		branch		method	
	α	β	α	β	α	β
assert-core	0.05	0.45	0.15	0.65	0.35	0.85
jasmine-maven-plugin	0.5	0.85	0	0.8	0.4	0.7
java-apns	0.25	1	0	1	0.05	1
jopt-simple	0.45	1	0.35	1	0.45	1
metrics-core	0.15	0.55	0	0.35	0.05	0.5
scribe-java	0.25	0.85	0	0.9	0.1	0.75
vraport	0.15	0.65	0	0.9	0.45	1

6 ADDITIONAL-SPREADING VERSUS OTHERS TCP TECHNIQUES

In this section, we present two empirical studies designed to: i) compare *Additional-Spreading* to other well-known TCP techniques; and ii) show its configurable power. To guide this investigation, we set the following research questions:

- **RQ3:** How does *Additional-Spreading* compare to traditional TCP strategies?
- **RQ4:** Can *Additional-Spreading* favor different testing goals?

6.1 Comparing Additional-Spreading

For RQ3, we design an study that reuses most artifacts from Section 5 with two main differences: i) we do not include the **la4j** project. As discussed in Section 5.3.1, we consider it an outlier project; and ii) we compare *Additional-Spreading* to the two TCP strategies that presented the best results for each testing goal metrics in previous work [28] - *Additional* (best APFD results), and *Total* (best M-Spreading results).

As *Additional-Spreading* requires input values (α and β), in this investigation we use the “balanced” values from Table 4. It is important to remember that those values are not fixed and represent only indicators to the testers of each project.

The procedure of this study goes as follows: each test suite is prioritized using nine TCP techniques (*Additional-statement*, *Additional-method*, *Additional-branch*, *Total-statement*, *Total-method*, *Total-branch*, *Add-spreading-statement*, *Add-spreading-method*, and *Add-spreading-branch*). Results are collected considering the fault groups created, and the two metrics (APFD and M-Spreading).

To answer RQ3, we plotted the box-plots for each metric (e.g., Figures 6 and 7⁴). Then, we applied the Levene test and Welch’s ANOVA that showed the techniques do not behave the same. Finally, we used the *post-hoc* (Turkey’s(*HSD*)) test and ranked them according to each metric (Table 5).

As we can see, the top positions remain the same as found by Torres et al. [28], *Additional* for APFD and *Total* for M-Spreading. However, *Additional-Spreading*, in most of the cases, takes intermediary positions. This was expected since we used balanced values for α and β . However, considering only the APFD ranking, we can see that *Additional-Spreading* performed worst than some of the *Total* variations. That was a surprising result, which leads us to infer that in fact, the chosen parameters need careful adjusting. However, later we discuss that *Additional-Spreading* can get closer to *Additional* results with the proper configuration (Section 6.2).

Table 5: Ranking of TCP techniques for all prioritizations. $A > B$ means technique A produced better results than B.

APFD	M-Spreading
Additional-statement > Additional-branch > Additional-method > Total-branch > Total-statement > Additional-Spreading-statement > Additional-Spreading-branch > Total-method > Additional-Spreading-method	Total-statement = Total-branch > Total-method > Additional-Spreading-statement > Additional-Spreading-method > Additional-Spreading-branch > Additional-statement = Additional-branch > Additional-method

6.2 Different Testing Goals

Regarding RQ4, to evidence the adaptive potential of *Additional-Spreading* for different testing goals, we run a different empirical study. In this study, we apply near limit values for α and β in order to favor either fastening fault detection (APFD) or better test case grouping (M-Spreading).

⁴The other plots can be found in our website

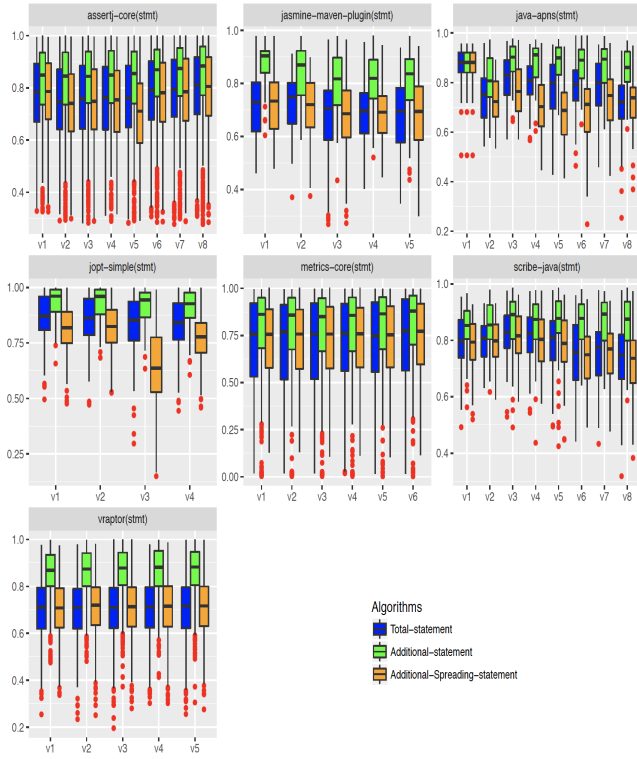


Figure 6: APFD distributions for the statement prioritizations.

For this study, we reuse the infrastructure of the previous study – Section 6.1 (projects, test suites, faults, techniques), and we run *Additional-Spreading* by using two sets of parameters $((\alpha_1, \beta_1), (\alpha_2, \beta_2))$. Those values were selected based on the results distributions from Section 5. We select the parameters combination that led to near best values of APFD (α_1, β_1) , and M-Spreading (α_2, β_2) . Thus, for each project, we work with two configurations. By near best, we mean the third quartile values. We decided to use the third quartile, instead of the fourth, to avoid working with extreme scenarios. For instance, for the *jopt-simple* project using *Additional-Spreading-statement*, we selected $\alpha = 0.75$ and $\beta = 0.3$ to favor fastening fault detection (APFD). On the other hand, when favoring better test grouping (M-Spreading), we used $\alpha = 0.35$ and $\beta = 0.75$. In this study, our goal is to show the configuration power of *Additional-Spreading*. Our idea is that a single technique, when properly configured, may be used for different testing goals.

Figures 8 and 9 present the results of this investigation for *Additional-Spreading-statement*. Similar trends were found for method and branch. As we can see, for APFD (Figure 8), *Additional-Spreading's* behavior was very similar to the *Additional*. Their medians were always similar with box overlapping. It is important to highlight that *Additional* was the strategy with best APFD results according to [28]. On the other hand, we observe an opposite trend when we evaluate the results using the configuration that favors M-Spreading (Figure 9). The *Additional-Spreading* results were mostly similar

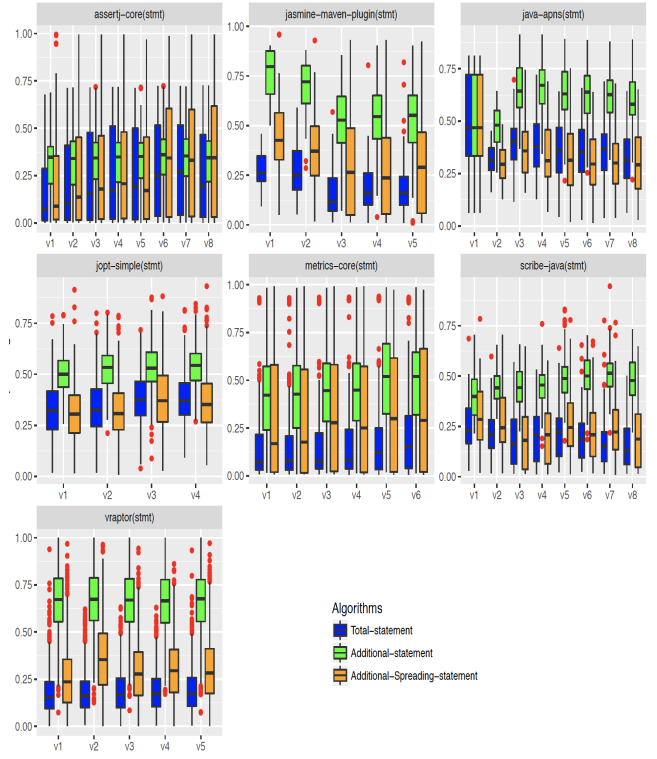


Figure 7: M-Spreading distributions for the statement prioritizations.

to the *Total* ones – *Total* was the best strategy for reducing the spreading of fault revealing test cases, according to [28].

Therefore, we can state that *Additional-Spreading* is a prioritization strategy that can be used to favor two different testing goals, when properly configured: fastening fault detection – APFD similar to *Additional*; or reduce the spreading of failing test cases – M-Spreading similar to *Total*.

7 GUIDELINES FOR ADDITIONAL-SPREADING

Here, we suggest a few guidelines to help testers better configure *Additional-Spreading*:

- If one decides to favor fast fault detection, we suggest applying low Similarity (α) and Cut (β) values. Our results show that with this configuration *Additional-Spreading* gets closer to the *Additional* prioritization (better APFD).
- If one decides to favor fault localization, we suggest applying high values for both α and β . Our results show that with this configuration *Additional-Spreading* gets closer to the *total* prioritization (better M-Spreading).
- If a balance between fast fault detection and help for fault localization is desired, we believe one should run a side investigation to figure the best configuration for the system under test. Our results show that it is possible to find a balanced configuration for *Additional-Spreading* by varying the values for α and β .

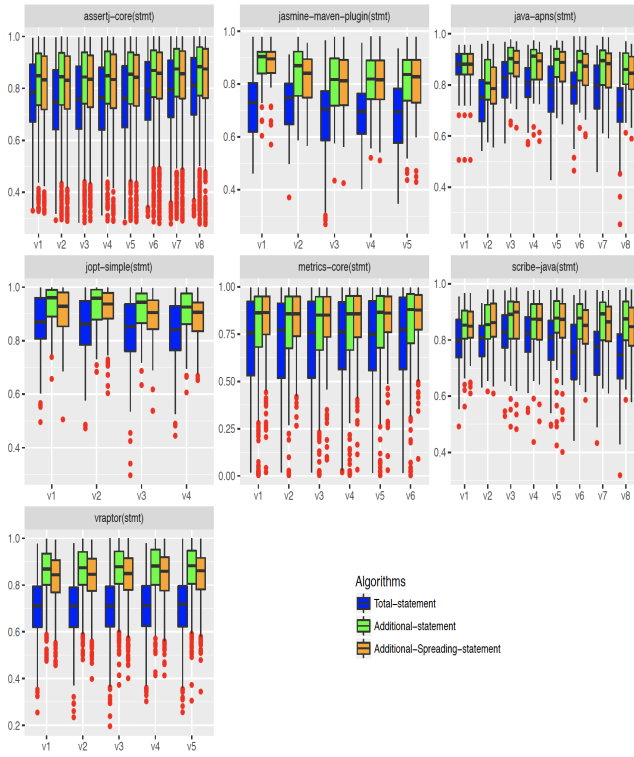


Figure 8: APFD when Additional-Spreading favors fastening fault detection.

8 THREATS TO VALIDITY

Our results do not generalize beyond the projects used in our study. However, due to the variability of the characteristics and context of the projects, we believe they are good representatives of the universe of open-source Java programs. Moreover, our data set of projects and TCP implementations were extracted from other works [17, 28]. A possible threat to our results is the fact we used seeded faults. However, seeded faults have been widely used in prioritization studies [8, 19]. Moreover, other researchers have found mutants as a good representative of real faults [5, 11, 31]. Finally, we could have used a different set of evaluation metrics. However, we opted to use APFD and *M-Spreading* because the first is the most well-known metric for prioritization and the second brings a different evaluation aspect.

9 RELATED WORK

The body of research on TCP is extensive. There is a wide range of TCP techniques. They vary by heuristics and/or context of use. Singh et al.'s present a literature review [27] with 106 TCP techniques from eight categories: coverage-based, modification-based, fault-based, requirement-based, history-based, genetic-based, composite approaches, and others. Though extensive, Khatibsyarhini et al. [14] discuss that TCP techniques are still subject to improvement.

The most well-known TCP techniques use coverage data as prioritization criteria (e.g., *Total* and the *Additional* [25]). Zhang et al. [31] propose a unified model that combines elements from both

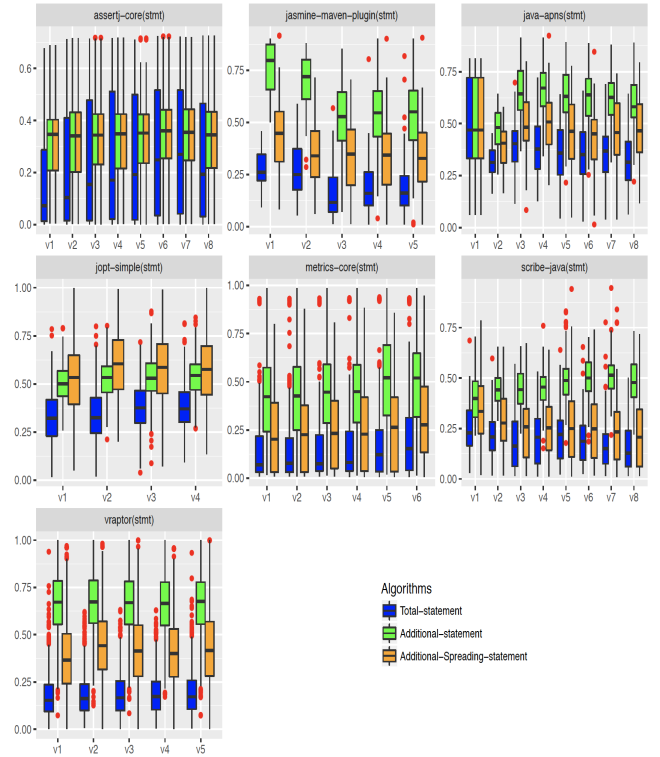


Figure 9: M-Spreading when Additional-Spreading favors reducing spreading.

strategies. Though with similar purposes, Zhang et al.'s approach use probabilities, while *Additional-Spreading* is configurable using two simple parameters (similarity and cut).

Rothermel et al. investigate the influence of different coverage criteria for prioritization [24] and how TCP works with different faults [25]. Jiang et al [12] investigate how TCP techniques may impact fault localization. They found that coverage-based and random techniques can be more effective and corroborates with our claims that prioritization may support testing activities beyond fault detection (e.g., fault understanding, fault localization).

Regarding metrics for evaluating prioritization, APFD is the most popular [29]. However, other metrics can be found: APCC (Average Percentage of λ -wise Combinations Covered)[10] measures the interaction rate of strength λ , where the strength is the level of interaction between a test case and a code unit; PTR (Problem Tracking Reports) [22] calculates the percentage of prioritized cases that need to be executed for detecting all faults. *M-Spreading* is still little explored [28]. We believe this metric provides a different aspect of analysis and can be used combined with others.

10 CONCLUDING REMARKS

Test case prioritization is often used to fasten fault detection. However, other testing activities should also be considered. For instance, a better grouping of fault revealing test cases may help a tester understand and locate a given fault. APFD and *M-Spreading* are

two metrics used for evaluating prioritization results. The first measures how fast the faults are detected, while the latter analyses the grouping of related tests. In this paper, we present *Additional-Spreading* an adaptive prioritization strategy that enables a tester to favor a specific testing goal: fastening fault detection (good APFD) or reducing the spreading of fault revealing test cases (good M-Spreading). We ran a series of empirical studies that evidenced *Additional-Spreading* configuration power, and show how it can generate results similar to two traditional techniques (Total and Additional), when properly used. Moreover, we provided guidelines on how to use it.

As future work, we plan to expand our studies by including new metrics, projects, and real faults. Moreover, we plan to run a case study in an active project to better understand how testers would use *Additional-Spreading* in practice and evaluate their perspectives. Finally, we intend to run a study to understand how the system and suite's characteristics may influence the spreading of failing test cases.

ARTIFACT AVAILABILITY

The artifacts used in our studies and a detailed version of our results are available in our website⁵

ACKNOWLEDGMENTS

This research was partially supported by the National Council for Scientific and Technological Development (CNPq)/Brazil (process 429250/2018-5).

REFERENCES

- [1] Everton LG Alves, Patrícia DL Machado, Tiago Massoni, and Miryung Kim. 2016. Prioritizing test cases for early detection of refactoring faults. *Software Testing, Verification and Reliability* 26, 5 (2016), 402–426.
- [2] Everton L.G. Alves, Tiago Massoni, and Patrícia Duarte de Lima Machado. 2017. Test coverage of impacted code elements for detecting refactoring faults: An exploratory study. *Journal of Systems and Software* 123 (2017), 223 – 238. <https://doi.org/10.1016/j.jss.2016.02.001>
- [3] Ana Emilia Victor Barbosa Coutinho, Emanuela Gadelha Cartaxo, and Patrícia Duarte de Lima Machado. 2016. Analysis of distance functions for similarity-based test suite reduction in the context of model-based testing. *Software Quality Journal* 24, 2 (2016), 407–445.
- [4] Thomaz Diniz, Everton LG Alves, Anderson GF Silva, and Wilkerson L Andrade. 2019. Reducing the Discard of MBT Test Cases using Distance Functions. In *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*. 337–346.
- [5] Hyunsook Do, Siavash Mirarab, Ladan Tahvildari, and Gregg Rothermel. 2010. The effects of time constraints on test case prioritization: A series of controlled experiments. *IEEE Transactions on Software Engineering* 36, 5 (2010), 593–617.
- [6] Martin Fowler. 2018. *Refactoring - Improving the Design of Existing Code* (2 ed.). Addison-Wesley.
- [7] Dan Hao, Lu Zhang, and Hong Mei. 2016. Test-case prioritization: achievements and challenges. *Frontiers of Computer Science* 10, 5 (2016), 769–777.
- [8] Dan Hao, Lu Zhang, Lei Zang, Yanbo Wang, Xingxia Wu, and Tao Xie. 2016. To be optimal or not in test-case prioritization. *IEEE Transactions on Software Engineering* 42, 5 (2016), 490–505.
- [9] Anna Huang. 2008. Similarity measures for text document clustering. In *Proceedings of the sixth new zealand computer science research student conference (NZCSRSC2008)*, Christchurch, New Zealand, Vol. 4. 9–56.
- [10] Rubing Huang, Yunan Zhou, Weiwen Zong, Dave Towey, and Jinfu Chen. 2017. An Empirical Comparison of Similarity Measures for Abstract Test Case Prioritization. In *Computer Software and Applications Conference (COMPSAC)*, 2017 IEEE 41st Annual, Vol. 1. IEEE, 3–12.
- [11] Bo Jiang, Zhenyu Zhang, Wing Kwong Chan, and TH Tse. 2009. Adaptive random test case prioritization. In *Proceedings of the 2009 IEEE/ACM International Conference on Automated Software Engineering*. IEEE Computer Society, 233–244.
- [12] B. Jiang, Z. Zhang, T. H. Tse, and T. Y. Chen. 2009. How Well Do Test Case Prioritization Techniques Support Statistical Fault Localization. In *2009 33rd Annual IEEE International Computer Software and Applications Conference*, Vol. 1. 99–106. <https://doi.org/10.1109/COMPSAC.2009.23>
- [13] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. ACM, 437–440.
- [14] Muhammad Khatibsyarhini, Mohd Adham Isa, Dayang N.A. Jawawi, and Rooster Tumeng. 2018. Test case prioritization approaches in regression testing: A systematic literature review. *Information and Software Technology* 93 (2018), 74 – 93. <https://doi.org/10.1016/j.infsof.2017.08.014>
- [15] Vladimir Isosifovich Levenshtein. 1966. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady* 10, 8 (feb 1966), 707–710. Doklady Akademii Nauk SSSR, V163 No4 845-848 1965.
- [16] Zheng Li, Mark Harman, and Robert M Hierons. 2007. Search algorithms for regression test case prioritization. *IEEE Transactions on software engineering* 33, 4 (2007).
- [17] Yafeng Lu, Yiling Lou, Shiyang Cheng, Lingming Zhang, Dan Hao, Yangfan Zhou, and Lu Zhang. 2016. How does regression test prioritization perform in real-world software evolution?. In *Software Engineering (ICSE)*, 2016 IEEE/ACM 38th International Conference on. IEEE, 535–546.
- [18] Andre Hasudungan Lubis, Ali Ikhwani, and Phak Len Eh Kan. 2018. Combination of levenshtein distance and rabin-karp to improve the accuracy of document equivalence level. *International Journal of Engineering & Technology* 7, 2.27 (2018), 17–21.
- [19] Qi Luo, Kevin Moran, and Denys Poshyvanyk. 2016. A large-scale empirical comparison of static and dynamic test case prioritization techniques. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, 559–570.
- [20] Suphakit Niwattanakul, Jatsada Singthongchai, Ekkachai Naenudorn, and Supachanun Wanapu. 2013. Using of Jaccard coefficient for keywords similarity. In *Proceedings of the international multicongference of engineers and computer scientists*, Vol. 1. 380–384.
- [21] Francisco G Oliveira Neto, Richard Torkar, and Patrícia DL Machado. 2016. Full modification coverage through automatic similarity-based test case selection. *Information and Software Technology* 80 (2016), 124–137.
- [22] R Pradeepa and Vimala K Devi. 2013. Effectiveness of Testcase Prioritization using APFD Metric: Survey. In *IJCA Proceedings on International Conference on Research Trends in Computer Technologies*. 1–4.
- [23] Gregg Rothermel and Mary Jean Harrold. 1997. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 6, 2 (1997), 173–210.
- [24] Gregg Rothermel, Roland H Untch, Chengyun Chu, and Mary Jean Harrold. 1999. Test case prioritization: An empirical study. In *Software Maintenance, 1999.(ICSM'99) Proceedings. IEEE International Conference on*. IEEE, 179–188.
- [25] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. 2001. Prioritizing test cases for regression testing. *IEEE Transactions on software engineering* 27, 10 (2001), 929–948.
- [26] Thomas A Runkler and James C Bezdek. 2000. Automatic keyword extraction with relational clustering and Levenshtein distances. In *Ninth IEEE International Conference on Fuzzy Systems. FUZZ-IEEE 2000 (Cat. No. 00CH37063)*, Vol. 2. IEEE, 636–640.
- [27] Yogesh Singh, Arvinder Kaur, Bharti Suri, and Shweta Singhal. 2012. Systematic literature review on regression test prioritization techniques. *Informatica* 36, 4 (2012).
- [28] Wesley NM Torres, Everton LG Alves, and Patrícia DL Machado. 2019. An Empirical Study on the Spreading of Fault Revealing Test Cases in Prioritized Suites. In *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 1. IEEE, 129–138.
- [29] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability* 22, 2 (2012), 67–120.
- [30] Shin Yoo, Mark Harman, and David Clark. 2013. Fault localization prioritization: Comparing information-theoretic and coverage-based approaches. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 22, 3 (2013), 19.
- [31] Lingming Zhang, Dan Hao, Lu Zhang, Gregg Rothermel, and Hong Mei. 2013. Bridging the gap between the total and additional test-case prioritization strategies. In *Proceedings of the 2013 International Conference on Software Engineering*. IEEE Press, 192–201.
- [32] Lingming Zhang, Miryung Kim, and Sarfraz Khurshid. 2012. FaultTracer: a change impact and regression fault analysis tool for evolving Java programs. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*. ACM, 40.

⁵<https://github.com/WesleyMarques/tcp-experiment/>